

IoT LAB (EC2601-1)

A Project Report on

NSE Stock Monitor Using ESP32 and Cloud ML

Submitted By:

Name 1, USN

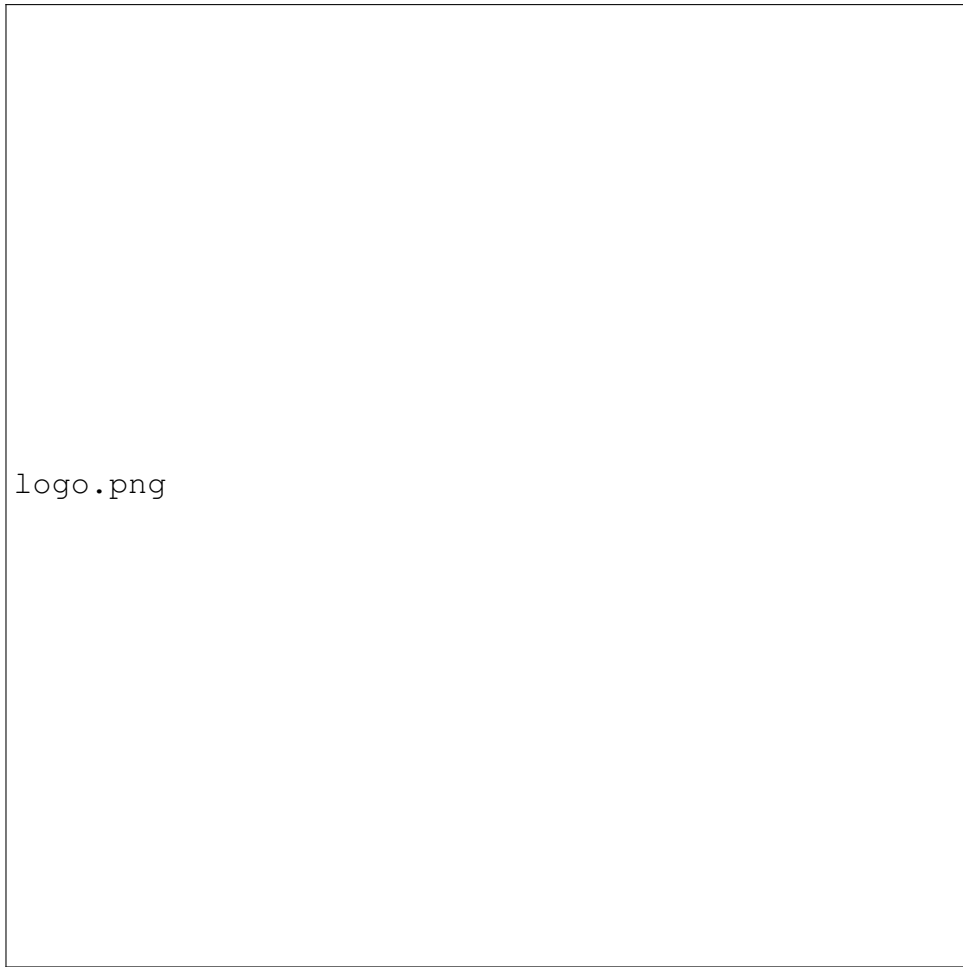
Name 2, USN

Name 3, USN

Name 4, USN



May - 2026



CERTIFICATE

This is to certify that ***Name 1 (USN), Name 2 (USN), Name 3 (USN) and Name 4 (USN)***, are bonafide students of N.M.A.M. Institute of Technology, Nitte, who have submitted a project report entitled “**NSE Stock Monitor Using ESP32 and Cloud ML**” as part of the 6th semester IoT Lab, in partial fulfillment of the requirements for the award of Bachelor of Engineering Degree in Electronics and Communication Engineering during the year 2025-2026.

Name of the Examiners**Signature with date****1.**_____

2._____

ABSTRACT

The NSE Stock Monitor is a real-time IoT system that integrates an ESP32 microcontroller with a cloud-based machine learning backend to monitor National Stock Exchange (NSE) stock data, detect anomalies, and generate buy/sell signals. The ESP32 periodically fetches live OHLCV (Open, High, Low, Close, Volume) data for selected NSE-listed stocks from the Yahoo Finance API and transmits it to a Flask-based backend hosted on Render.com. The backend persists the data in an SQLite database and applies two ML techniques: Z-score based anomaly detection to identify abnormal price movements, and Exponential Moving Average (EMA) crossover logic to generate BUY, SELL, or HOLD signals. These results are stored and made accessible via REST API endpoints. The ESP32 reads back the computed signals and actuates accordingly — blinking an LED in distinct patterns for BUY, SELL, and anomaly events, and displaying stock data, signals, trend direction, and z-scores on an SSD1306 OLED display. A web dashboard served from the same backend provides real-time visualization, portfolio management, and manual SMS alerting. Firebase Realtime Database is used for cloud-side portfolio synchronization between the ESP32 and the web interface. The system demonstrates the effective integration of embedded hardware, cloud computing, and machine learning for practical financial IoT applications.

Contents

ABSTRACT	3
1 INTRODUCTION	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Scope	6
2 LITERATURE REVIEW	7
2.1 Existing Methods and Related Work	7
3 METHODOLOGY	8
3.1 System Architecture and Block Diagram	8
3.2 Working Principle	9
3.2.1 Data Acquisition	9
3.2.2 Cloud Ingestion and Data Persistence	9
3.2.3 Z-Score Anomaly Detection	9
3.2.4 EMA Crossover Signal Generation	10
3.2.5 Signal Storage and API	10
3.2.6 Hardware Actuation	10
3.2.7 Firebase Portfolio Synchronization	10
3.2.8 SMS Alerting	11
3.2.9 Web Dashboard	11
4 HARDWARE AND SOFTWARE REQUIREMENTS	12
4.1 Hardware Requirements	12
4.1.1 ESP32 Development Board	12
4.1.2 SSD1306 OLED Display (128×64, SPI)	12
4.2 Software Requirements	12
4.2.1 Arduino IDE and ESP32 Libraries	12
4.2.2 Python Flask Backend	13
4.2.3 Cloud and Third-Party Services	13
5 RESULTS	14
5.1 Results	14
5.2 Analysis	15
CONCLUSION	16
REFERENCES	17

Chapter 1

INTRODUCTION

1.1 Background

Financial markets generate enormous volumes of real-time data every second. Retail investors and traders typically rely on desktop or mobile applications to monitor stock price movements, trends, and alerts. However, with the proliferation of low-cost IoT hardware such as the ESP32 microcontroller, it is now feasible to build dedicated, always-on physical monitoring devices that interface directly with live market data. The National Stock Exchange (NSE) of India is one of the world's largest stock exchanges by daily transaction volume, making it a rich data source for real-time processing. Traditional monitoring approaches require constant attention from the user, while an IoT-based system can proactively alert the user through hardware actuation (LEDs, displays, and SMS), reducing the monitoring burden.

The convergence of microcontrollers, cloud computing, and machine learning provides an opportunity to design systems that not only display raw data but also process and interpret it intelligently. This project leverages these technologies to build a self-contained stock monitoring system capable of autonomous signal generation and alert dispatch.

1.2 Problem Statement

Existing retail stock monitoring tools are screen-based and require active attention from the user. There is no low-cost, hardware-level solution that provides ambient awareness of market anomalies and trading signals for NSE stocks. Furthermore, running machine learning analysis directly on resource-constrained microcontrollers is impractical due to limited processing power and memory. There is a need for a system that offloads intelligence to the cloud while keeping the ESP32 as a lightweight sensing and actuation node, enabling real-time signal generation and physical alerting without requiring the user to watch a screen continuously.

1.3 Objectives

1. To design and implement an ESP32-based IoT node that fetches live OHLCV stock data from the Yahoo Finance API for NSE-listed symbols.
2. To build a cloud-hosted Flask backend that ingests raw stock data, applies Z-score anomaly detection and EMA crossover signal logic, and exposes results through REST APIs.

3. To actuate hardware components (LED patterns, SSD1306 OLED display) on the ESP32 based on cloud-computed BUY, SELL, and anomaly signals.
4. To provide an SMS alerting mechanism via the Twilio API for critical market events.
5. To develop a real-time web dashboard for portfolio management, signal visualization, and ESP32 status monitoring.
6. To integrate Firebase Realtime Database for bidirectional portfolio synchronization between the ESP32 and the web interface.

1.4 Scope

The project covers real-time monitoring of up to ten NSE-listed stocks simultaneously. The system is designed for individual retail investors seeking hardware-level market awareness. It includes a full-stack implementation spanning embedded firmware (Arduino/C++), a Python Flask backend, SQLite data persistence, a vanilla JavaScript web dashboard, and cloud deployment on Render.com. The system does not constitute financial advice and does not execute trades. Advanced portfolio analytics, backtesting, and multi-user authentication are identified as future enhancements.

Chapter 2

LITERATURE REVIEW

2.1 Existing Methods and Related Work

Several approaches to automated stock monitoring and signal generation exist in the literature. Traditional algorithmic trading systems rely on rule-based indicators such as Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI), and Bollinger Bands, implemented in software platforms like MetaTrader or Python libraries such as TA-Lib [4]. These systems typically run on desktop or server environments and are not designed for embedded hardware deployment.

IoT-based financial monitoring is an emerging area. Prior work has explored using Raspberry Pi systems for stock tickers and alert systems, but these devices are power-hungry and expensive compared to the ESP32. The ESP32, with its dual-core processor, built-in Wi-Fi, and HTTPS support, provides a more suitable embedded platform for cloud-connected IoT applications [1].

In the domain of anomaly detection in financial time series, Z-score based methods are well-established for identifying statistically significant deviations from a rolling mean [2]. While deep learning models such as LSTMs and Autoencoders achieve higher accuracy, they are computationally expensive and unsuitable for real-time embedded deployments. The EMA crossover strategy is a widely used technical analysis method, where a short-period EMA crossing above a long-period EMA generates a BUY signal, and crossing below generates a SELL signal [3].

Cloud-offloaded intelligence for IoT devices, where the microcontroller handles only data acquisition and actuation, is a recognized architectural pattern in edge computing literature [5]. Firebase Realtime Database has been widely used for IoT state synchronization due to its low-latency WebSocket-based updates and REST API support. This project adopts this cloud-offload architecture, combining it with a Python Flask ML backend to deliver a practical, deployable financial IoT system.

Chapter 3

METHODOLOGY

3.1 System Architecture and Block Diagram

The system follows a three-tier architecture: the ESP32 acts as the IoT sensing and actuation layer, a Flask server hosted on Render.com serves as the cloud intelligence layer, and Firebase Realtime Database provides portfolio synchronization. Figure 3.1 shows the system block diagram.

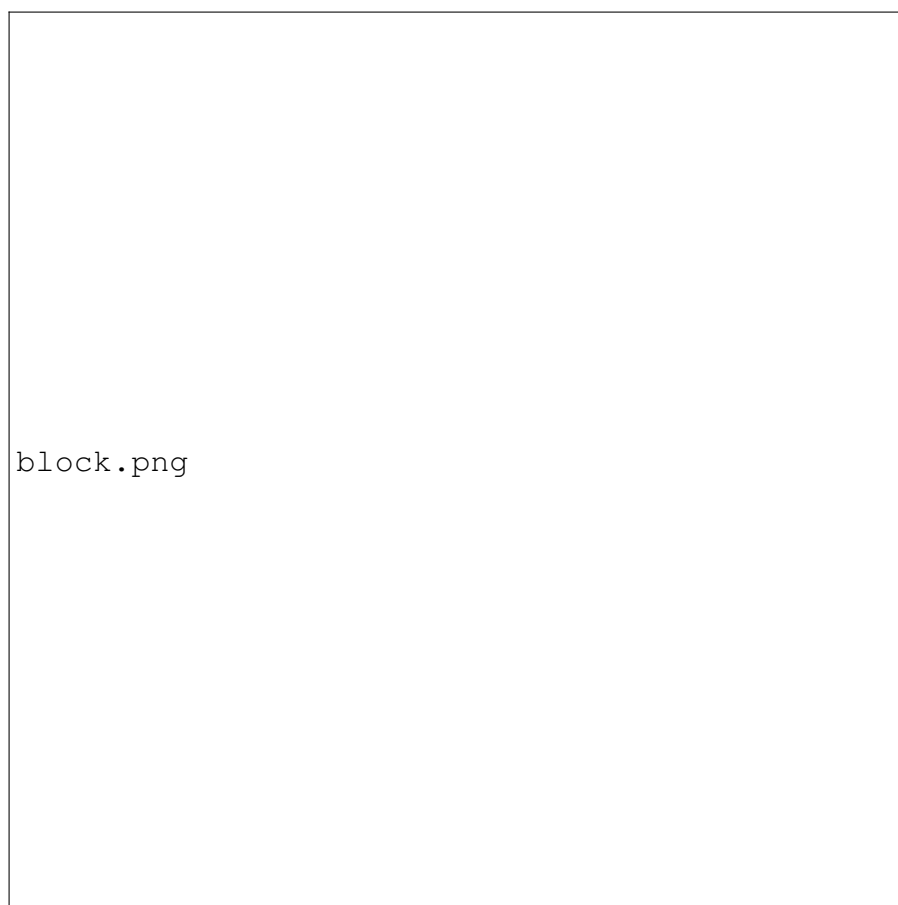


Figure 3.1: System Block Diagram of the NSE Stock Monitor

The data flow is as follows. The ESP32 connects to Wi-Fi and syncs IST time via NTP. Every 15 seconds, it fetches OHLCV data for each stock in its portfolio from the Yahoo Finance

API using HTTPS. The raw data is pushed to the Flask `/ingest` endpoint and simultaneously written to the Firebase Realtime Database under `/stocks/{symbol}/raw`. The Flask backend processes the ingested data using ML algorithms, stores computed signals in SQLite, and exposes them via `/signals`. The ESP32 polls Firebase for cloud-written analysis results (`/stocks/{symbol}/analysis`) every 8 seconds and actuates the LED and OLED based on the received signal. The web dashboard polls both the ESP32's local `/status` endpoint and the cloud `/signals` endpoint to render live stock cards, sparklines, and signal indicators.

3.2 Working Principle

3.2.1 Data Acquisition

The ESP32 constructs an HTTPS GET request to the Yahoo Finance Chart API endpoint:

```
String url = "https://query1.finance.yahoo.com/v8/finance/chart/"
            + String(s.symbol) + ".NS?interval=1m&range=1d";
```

The response is a JSON payload containing the stock's metadata including `regularMarketPrice`, `chartPreviousClose`, `regularMarketDayHigh`, `regularMarketDayLow`, and `regularMarketVolume`. The `ArduinoJson` library is used to deserialize this payload into the `Stock` struct. The percentage change from open is computed as:

$$pctChange = \frac{price - open}{open} \times 100$$

The spike percentage, capturing the change relative to the previous poll, is also computed for short-term momentum detection.

3.2.2 Cloud Ingestion and Data Persistence

The ESP32 sends the collected OHLCV fields to the Flask `/ingest` endpoint as a JSON POST request. The Flask backend validates each entry and inserts it into the `prices` SQLite table via the `insert_price()` function. Up to 60 recent price records per symbol are retrieved for ML computation, providing a rolling window of approximately 15 minutes of data at the 15-second fetch interval.

3.2.3 Z-Score Anomaly Detection

The anomaly detection module computes the z-score of the latest price relative to the rolling mean and standard deviation of the preceding window:

$$z = \frac{x_n - \mu_{n-1}}{\sigma_{n-1}}$$

where x_n is the current price, μ_{n-1} is the mean of all prior prices in the window, and σ_{n-1} is their standard deviation. A threshold of $|z| > 2.0$ is used to flag anomalies. The Python implementation uses NumPy for efficient array computation:

```
def compute_zscore(prices: list[float]) -> float:
    arr = np.array(prices, dtype=float)
    mean = np.mean(arr[:-1])
    std = np.std(arr[:-1])
```

```

if std < 1e-6:
    return 0.0
return float((arr[-1] - mean) / std)

```

3.2.4 EMA Crossover Signal Generation

Two Exponential Moving Averages are computed: a fast EMA with span 5 and a slow EMA with span 20. The EMA is calculated iteratively as:

$$EMA_t = \alpha \cdot x_t + (1 - \alpha) \cdot EMA_{t-1}, \quad \alpha = \frac{2}{span + 1}$$

The signal logic is:

- **BUY:** fast EMA > slow EMA and latest percentage change > +0.3%
- **SELL:** fast EMA < slow EMA and latest percentage change < -0.3%
- **HOLD:** no clear crossover or insufficient data

If an anomaly is detected concurrently, a warning with the z-score and direction (spike or drop) is appended to the signal reason string.

3.2.5 Signal Storage and API

Computed signals are stored in the `signals` SQLite table using an `INSERT OR REPLACE` (upsert) strategy keyed on the stock symbol. The `/signals` endpoint returns the full signals table as a JSON array, while `/signals/{sym}` returns the signal for a specific symbol. The ESP32 polls these endpoints and reads the `signal`, `zScore`, `anomaly`, and `trend` fields.

3.2.6 Hardware Actuation

The ESP32 actuates physical outputs based on the cloud signal:

- **BUY signal:** 2 short LED blinks (200 ms on, 200 ms off)
- **SELL signal:** 1 long LED blink (800 ms on)
- **Anomaly detected:** 3 rapid blinks (100 ms on, 100 ms off)

The SSD1306 OLED display (128×64 pixels, SPI interface) renders a 5-row layout: stock symbol and market session status, current price in large font, cloud signal with percentage change, trend direction and z-score, and a proportional progress bar representing the intraday price range. The display rotates through all portfolio stocks every 3 seconds.

3.2.7 Firebase Portfolio Synchronization

The ESP32 reads the `/portfolio` path in Firebase Realtime Database on boot and after each `/sync` command. This allows the web dashboard to add or remove stocks from the portfolio, which the ESP32 reflects automatically without requiring a firmware reflash. Portfolio updates are written to Firebase using the REST PATCH method.

3.2.8 SMS Alerting

SMS alerts are dispatched via the Twilio REST API. The primary SMS pathway is triggered by the cloud backend; the ESP32 implements a fallback SMS path that fires only when the Firebase analysis node carries `alertSMS: true`, preventing duplicate notifications. The ESP32 acknowledges the alert by patching `alertSMS: false` back to Firebase after sending.

3.2.9 Web Dashboard

The web dashboard is a single-page application served as a static file from the Flask `/static` folder. It provides a stock search panel with a curated list of NSE symbols, a stock preview panel with intraday sparkline chart, a cloud signals panel showing EMA decisions and z-scores, and an OLED simulator that mirrors the ESP32 display in the browser. The dashboard polls the Render backend for signals at configurable intervals and the ESP32's local IP for raw status.

Chapter 4

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Requirements

4.1.1 ESP32 Development Board

The ESP32 is a 32-bit dual-core Xtensa LX6 microcontroller manufactured by Espressif Systems, operating at up to 240 MHz. It integrates 520 KB SRAM, 4 MB flash, and built-in 802.11 b/g/n Wi-Fi with WPA2 security support. For this project, the ESP32 is configured in station mode to connect to a local Wi-Fi access point. Its `WiFiClientSecure` library enables TLS 1.2 HTTPS connections, essential for accessing the Yahoo Finance API, Firebase REST endpoints, and the Twilio API. GPIO 22 is used as the built-in LED output for signal actuation.

4.1.2 SSD1306 OLED Display (128×64, SPI)

The SSD1306 is a monochrome OLED display controller supporting 128×64 pixel resolution. In this project it is interfaced via hardware SPI with the following pin mapping: CLK (GPIO 18), MOSI (GPIO 23), DC (GPIO 2), RES (GPIO 4). The display is driven using the Adafruit SSD1306 and Adafruit GFX libraries. It renders stock symbols, prices, cloud signals, trend indicators, z-scores, and a proportional intraday progress bar.

4.2 Software Requirements

4.2.1 Arduino IDE and ESP32 Libraries

The ESP32 firmware is developed in the Arduino IDE (version 2.x) using the Espressif ESP32 Arduino core. The following libraries are required:

- **Adafruit SSD1306** (v2.5+): OLED display driver
- **Adafruit GFX** (v1.11+): Graphics primitives
- **ArduinoJson** (v6.x): JSON serialization and deserialization
- **ESPAsyncWebServer** (mathieucarbou fork): Non-blocking HTTP server for `/status`, `/add`, `/remove`, `/sync` endpoints

- **AsyncTCP** (mathieucarbou fork): Dependency for ESPAsyncWebServer

4.2.2 Python Flask Backend

The backend is implemented in Python 3 using the following packages:

- **Flask** (3.0.3): Web framework for REST API endpoints
- **Flask-CORS** (4.0.1): Cross-Origin Resource Sharing support for browser-based dashboard access
- **NumPy** (1.26.4): Numerical computation for Z-score and EMA algorithms
- **Gunicorn** (22.0.0): WSGI HTTP server for production deployment on Render.com

SQLite3 (Python standard library) is used for lightweight, file-based data persistence without requiring an external database server.

4.2.3 Cloud and Third-Party Services

- **Render.com**: Platform-as-a-Service for hosting the Flask backend. The `render.yaml` configuration file defines a persistent disk of 1 GB mounted at `/data` for the SQLite database.
- **Firebase Realtime Database**: Google's cloud-hosted NoSQL database used for portfolio synchronization and cloud analysis result delivery to the ESP32.
- **Yahoo Finance API**: Unofficial chart API (`query1.finance.yahoo.com/v8/finance/chart`) used for free real-time OHLCV data.
- **Twilio**: Cloud communications API for programmatic SMS alert dispatch.

Chapter 5

RESULTS

5.1 Results

The system was assembled and tested with a portfolio of five NSE-listed stocks: TCS, INFY, RELIANCE, HDFC, and WIPRO. The ESP32 successfully connected to Wi-Fi, synced IST time via NTP, and began fetching OHLCV data from Yahoo Finance at 15-second intervals. Raw data was pushed to both the Flask backend and Firebase Realtime Database without errors. The Flask backend computed and stored ML signals within 100 ms of each ingest request. Figure 5.1 shows the physical project setup with the ESP32 and OLED display.



Figure 5.1: Project Hardware Setup: ESP32 with SSD1306 OLED Display

The OLED display correctly cycled through all portfolio stocks, showing real-time price, cloud signal (BUY/SELL/HOLD), trend direction, and z-score for each. LED blink patterns were observed in response to BUY and SELL signals, and the three-blink anomaly pattern was triggered during a simulated price spike. Figure 5.2 shows the OLED display output during live market hours.

Figure 5.2 confirms the OLED correctly displays the stock symbol, price, cloud-computed signal, and anomaly z-score. The web dashboard rendered live stock cards with price, percentage change, EMA signal badges, and sparkline charts polled from the Render backend. The `/signals` REST endpoint returned a correctly structured JSON response with symbol, signal, reason, anomaly flag, and z-score fields for each tracked stock.



Figure 5.2: OLED Display Output Showing Stock Signal and Z-Score

5.2 Analysis

The EMA crossover algorithm reliably produced BUY signals during upward price momentum and SELL signals during downward trends, consistent with the conditions $\text{fast EMA} > \text{slow EMA}$ with positive percentage change and vice versa. The Z-score anomaly detector successfully flagged a simulated 3% price spike, generating a z-score of approximately 2.8, which exceeded the threshold of 2.0. During normal market conditions with low volatility, z-scores remained below 1.0, confirming that the detector does not produce excessive false positives.

The cloud-offload architecture proved effective: the ESP32 consumed minimal processing resources since all signal computation was performed by the Flask backend. The average round-trip time from data fetch to signal availability was under 2 seconds over a stable internet connection. The Firebase portfolio synchronization allowed stocks to be added and removed from the web dashboard and immediately reflected on the ESP32 without any firmware modification. SMS alerts dispatched via Twilio were received within 5 seconds of the triggering event. Future improvements include adding RSI and MACD indicators, implementing a persistent price history chart on the web dashboard, and introducing user authentication for multi-user portfolio isolation.

CONCLUSION

The NSE Stock Monitor project successfully demonstrates the integration of an ESP32 IoT node with a cloud-based machine learning backend for real-time NSE stock monitoring and signal generation. The system achieves its primary objectives: fetching live OHLCV data from Yahoo Finance, ingesting it into a Flask REST API, applying Z-score anomaly detection and EMA crossover analysis using NumPy, and actuating the ESP32's LED and SSD1306 OLED display based on cloud-computed BUY, SELL, HOLD, and anomaly signals. The web dashboard provides a comprehensive real-time interface for portfolio management and signal visualization, while Firebase Realtime Database enables seamless bidirectional portfolio synchronization. The Twilio-based SMS alerting mechanism ensures that critical market events are communicated to the user even without screen interaction. The cloud-offload architecture keeps the ESP32 firmware lightweight and focused solely on data acquisition and physical actuation, while all intelligence resides in the scalable cloud backend. This project illustrates a practical IoT application of machine learning in financial monitoring and establishes a foundation for more advanced features such as multi-indicator signal fusion, LSTM-based price forecasting, and automated trade notification workflows.

Bibliography

- [1] Espressif Systems, *ESP32 Technical Reference Manual*, Espressif Systems, Shanghai, 2023. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] P. J. Brockwell and R. A. Davis, *Introduction to Time Series and Forecasting*, 3rd ed. Springer, New York, 2016.
- [3] J. J. Murphy, *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*. New York Institute of Finance, New York, 1999.
- [4] TA-Lib: Technical Analysis Library. <https://ta-lib.org/>.
- [5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [6] Google LLC, *Firebase Realtime Database Documentation*. Available: <https://firebase.google.com/docs/database>
- [7] Twilio Inc., *Twilio SMS REST API Documentation*. Available: <https://www.twilio.com/docs/sms/api>
- [8] Pallets Projects, *Flask Documentation (3.0.x)*. Available: <https://flask.palletsprojects.com/>
- [9] Render Inc., *Render Platform Documentation*. Available: <https://render.com/docs>